

PYTHON DATA SCIENCE

Weekend Lab

A hands-on consolidation guide for NumPy, Pandas, Matplotlib, Seaborn, and Jupyter

9 guided sessions	27 original labs
1 integrated capstone	Complete answer key

Designed for a learner preparing for AI application engineering

Original curriculum aligned to Python Data Science Handbook, 2nd Edition

Table of contents

Orientation and notebook workflow	4
Day 1 - NumPy and data manipulation	6
Session 1: NumPy fundamentals	6
Session 2: NumPy advanced operations	8
Session 3: NumPy advanced indexing	10
NumPy review lab	12
Day 2 - Pandas and data analysis	13
Session 4: Pandas fundamentals	13
Session 5: Pandas advanced operations	15
Session 6: Grouping and aggregation	17
Day 3 - Visualization	19
Session 7: Matplotlib fundamentals	19

TWO PACES, ONE CURRICULUM

The prompt combines a 12-hour calendar with nine modules described as 2-3 hours each. Use the **core route** for the supplied Saturday-Sunday-Monday schedule, or the **mastery route** (18-24 hours) to complete every exercise and extension.

Contents and coverage map

Session 8: Matplotlib customization	21
Session 9: Advanced visualization	23
Final capstone: support operations intelligence	25
Answer key	28
Quick reference	34
Resources and next steps	36
Index	37

Concept family	Primary lab	What you will produce
IPython / Jupyter	All sessions	Reproducible, timed notebook workflow
NumPy arrays and dtypes	1	A validated 3D sensor cube
Ufuncs, aggregates, broadcasting	2	A vectorized pricing and anomaly engine
Masks, fancy indexing, sorting	3	A ranked event investigation table
Pandas objects and selection	4	A clean service-ticket dataset
MultilIndex, concat, merge, join	5	A relational operations dataset
GroupBy and pivot tables	6	A regional performance report
Core Matplotlib	7	Line, scatter, contour, and density figures
Plot customization	8	A publication-quality annotated figure
3D and Seaborn	9	A compact analytical dashboard
End-to-end integration	Capstone	A decision-ready support operations brief

Orientation and notebook workflow

This lab turns familiar concepts into a single working habit: **question -> data -> array/table operation -> visual evidence -> decision**. You will work with original synthetic datasets that resemble sensor, commerce, and support-operations data. The values are invented, so the notebook is safe to rerun and share.

Learning goals

- Choose NumPy for fast homogeneous numerical computation and Pandas for labeled, heterogeneous tables.
- Express array operations with vectorization, broadcasting, masks, and deliberate axis choices.
- Build defensible joins and split-apply-combine analyses with explicit validation.
- Design plots whose scales, encodings, annotations, and layout answer a real question.
- Produce a restartable notebook whose results survive a clean kernel run.

Prerequisites and tools

No background beyond the covered chapters is assumed. Use Python 3.10+ in JupyterLab or Jupyter Notebook with NumPy, Pandas, Matplotlib, and Seaborn. The code avoids external downloads and runs from a fresh kernel.

```
# Run once in a terminal if needed
python -m pip install jupyterlab numpy pandas matplotlib seaborn

# Launch
jupyter lab
```

Preflight cell

```
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

rng = np.random.default_rng(20250720)
pd.set_option("display.max_columns", 20)
sns.set_theme(style="whitegrid", context="notebook")

print(sys.version.split()[0], np.__version__, pd.__version__)
```

JUPYTER PRACTICE LOOP

Use **?** and **??** to inspect objects; Tab to explore names; **%timeit** for microbenchmarks; **%run** for reusable scripts; **%history** when reconstructing work; **%debug** or **%pdb on** after an exception; and **Kernel -> Restart and Run All** before declaring a notebook complete.

How to work each session

- Predict first: write the expected shape, dtype, index, or visual pattern before running a cell.
- Code along: type the code; do not paste it. Change one argument and explain the result.
- Try it yourself: solve without the answer key, then add one assertion that proves correctness.
- Reflect: finish with two markdown sentences - what changed, and why it matters.

Weekend schedule

Day	Core block	What to prioritize
Saturday	3h Sessions 1-3	Code-alongs + challenge 3.3
Saturday	2h review	NumPy review lab + corrections
Sunday	3h Sessions 4-6	Joins, GroupBy, challenge 6.3
Sunday	2h Sessions 7-9	Plot code-alongs + dashboard sketch
Monday	2h capstone	Core path, narrative, restart-and-run

COMMON PITFALLS

- Running cells out of order hides dependencies. Restart and Run All is the real test.
- Timing one run is noisy. Prefer `%timeit` and compare equivalent work.
- A result without a shape, key, range, or row-count assertion is easy to misread.

Day 1: NumPy and data manipulation

NumPy is the numerical engine beneath much of the Python data stack. The day moves from memory and shape, through whole-array computation, to selective and structured access. Write shapes beside intermediate results; most NumPy bugs are shape misunderstandings.

DAY 1 CHECKPOINT

By the end, you can explain every axis in a 3D array, replace a scalar loop with a vectorized expression, and select/rank records without losing alignment.

Session 1: NumPy Fundamentals

DAY 1 LAB | 2-3 hours

Focus: arrays, dtypes, indexing, slicing, reshape, concatenate, split

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

1.1 Build arrays deliberately

An ndarray stores values of one dtype in a rectangular shape. Choose the shape as a data model: here, rows are sites and columns are hourly readings.

```
import numpy as np

readings = np.array([
    [18.2, 18.7, 19.1, 19.5],
    [21.0, 20.8, 20.4, 20.1],
    [16.8, 17.2, 17.9, 18.4],
], dtype=np.float64)

assert readings.shape == (3, 4)
print(readings.ndim, readings.size, readings.dtype)
print(np.zeros((2, 3)), np.arange(0, 12, 2))
print(np.linspace(0.0, 1.0, 5))
```

Use `arange` for integer-like steps and `linspace` when the number of evenly spaced values matters. Specify dtypes at boundaries: compact integers for codes, floating point for measured values.

1.2 Index. slice. and understand views

```
first_site = readings[0]          # shape (4,)
middle_hours = readings[:, 1:3]
last_hour_column = readings[:, -1:] # shape (3, 1), not (3,)

window = readings[:, :2]          # usually a view
window[0, 0] = -999.0
assert readings[0, 0] == -999.0

readings[0, 0] = 18.2
safe_window = readings[:, :2].copy()
safe_window[0, 0] = -1.0
assert readings[0, 0] == 18.2
```

SHAPE HABIT

Indexing with an integer removes an axis; slicing preserves it. Compare `a[:, 0]` with `a[:, 0:1]` before broadcasting or concatenating.

1.3 Reshape, concatenate, and split

```
sequence = np.arange(24)
cube = sequence.reshape(2, 3, 4) # days, sites, hours
assert cube[1, 2, 3] == 23

day_3 = (100 + np.arange(12)).reshape(1, 3, 4)
three_days = np.concatenate([cube, day_3], axis=0)

site_a, site_b, site_c = np.split(three_days, [1, 2], axis=1)
assert site_a.shape == site_b.shape == site_c.shape == (3, 1, 4)

features = np.column_stack([readings.mean(1), readings.max(1)])
assert features.shape == (3, 2)
```

TRY IT YOURSELF 1.1 - Shape audit

Create the integers 1 through 24, reshape them into (2, 3, 4), and extract the second day, first site, last two hours.

Deliverable: A shape-(2,) array equal to [15, 16]. Add two assertions.

Hint: Remember that Python indices start at zero.

TRY IT YOURSELF 1.2 - Sensor assembly

Create three shape-(4,) site arrays. Stack them as rows, append a fifth hour as a column, then split the result after rows 1 and 2.

Deliverable: One shape-(3, 5) array and three shape-(1, 5) arrays.

Hint: Use row_stack/vstack, column_stack, and split.

TRY IT YOURSELF 1.3 - Challenge: 3D operations

Build a (2 days, 3 stores, 4 products) revenue cube from values 100 through 123. Produce daily totals, store totals across both days, and a copy containing only products 1 and 3. Swap the day and store axes.

Deliverable: Four arrays with annotated shapes and a check that the original cube is unchanged.

Hint: Use sum(axis=...), fancy selection on the last axis, copy, and swapaxes.

COMMON PITFALLS

- Mixed Python values can silently promote a dtype, such as integers becoming floats or strings.
- A slice is commonly a view. Mutating it may mutate the source; use copy when isolation is intended.
- reshape preserves element count. Use -1 for exactly one inferred dimension.
- concatenate joins along an existing axis; stack creates a new axis.

Session 2: NumPy Advanced Operations

DAY 1 LAB | 2-3 hours

Focus: ufuncs, broadcasting, aggregation, vectorization, performance

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

2.1 Ufuncs replace scalar bookkeeping

Universal functions operate elementwise in compiled loops and compose naturally. Vectorization improves clarity first; measure performance second.

```
rng = np.random.default_rng(20250720)
latency_ms = rng.lognormal(mean=4.2, sigma=0.35, size=200_000)

slow = np.empty_like(latency_ms)
for i, value in enumerate(latency_ms):
    slow[i] = np.log1p(value) / np.sqrt(value)

fast = np.log1p(latency_ms) / np.sqrt(latency_ms)
np.testing.assert_allclose(slow, fast)

# In Jupyter, time equivalent work:
# %timeit np.log1p(latency_ms) / np.sqrt(latency_ms)
```

2.2 Broadcasting is shape arithmetic

Compare dimensions from the right. Dimensions are compatible when equal or when one is 1. Missing leading dimensions behave as 1. Make the intended alignment visible with `None` or `np.newaxis`.

```
sales = np.array([
    [12, 16, 11, 20],
    [10, 14, 18, 17],
    [15, 12, 13, 22],
], dtype=float) # stores x products: (3, 4)
price = np.array([9.0, 12.0, 7.5, 15.0]) # (4,)
store_factor = np.array([1.00, 0.95, 1.08])[None, :] # (3, 1)

revenue = sales * price * store_factor
assert revenue.shape == (3, 4)

centered = sales - sales.mean(axis=0, keepdims=True)
assert np.allclose(centered.mean(axis=0), 0)
```

2.3 Aggregations answer axis-specific questions

```
daily = np.array([
    [10.0, 12.0, np.nan, 9.0],
    [8.0, 11.0, 15.0, 10.0],
    [9.0, 13.0, 14.0, 11.0],
])

per_product_mean = np.nanmean(daily, axis=0)
per_store_total = np.nansum(daily, axis=1)
peak_position = np.nanargmax(daily)
peak_store, peak_product = np.unravel_index(peak_position, daily.shape)

print(per_product_mean, per_store_total)
print(peak_store, peak_product)
```

TRY IT YOURSELF 2.1 - Standardize columns

Generate a (100, 4) normal array. Standardize each column to mean 0 and standard deviation 1 without a loop.

Deliverable: A shape-(100, 4) array; assert column means are near 0 and standard deviations near 1.

Hint: Use `keepdims=True` for both aggregates.

TRY IT YOURSELF 2.2 - Pairwise distances

Given five 2D points, compute the full 5x5 Euclidean distance matrix with broadcasting.

Deliverable: A symmetric matrix with a zero diagonal.

Hint: Form differences with `points[:, None, :] - points[None, :, :]`.

TRY IT YOURSELF 2.3 - Challenge: vectorize billing

Replace a loop that charges each usage value at tiered rates: first 100 units at 0.12, next 200 at 0.18, and all remaining units at 0.25. Input is a 1D array of nonnegative usage values.

Deliverable: A vectorized function, tests at 0/100/300/450, and a `%timeit` comparison on 100,000 values.

Hint: Combine `np.minimum` and `np.clip`; calculate each tier separately.

COMMON PITFALLS

- Python `min/sum` and NumPy `min/sum` differ; use NumPy aggregations and specify axis explicitly.
- A broadcast that runs is not necessarily semantically correct. Label shapes in comments.
- Equality against NaN is always false. Use `np.isnan` or NaN-aware aggregations.
- A microbenchmark should compare equal outputs and exclude unrelated setup work.

Session 3: NumPy Advanced Indexing

DAY 1 LAB | 2-3 hours

Focus: comparisons, masks, boolean logic, fancy indexing, sorting, structured arrays

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

3.1 Masks express row-level rules

```
latency = np.array([82, 135, 95, 210, 178, 66, 142])
errors = np.array([0, 2, 0, 5, 1, 0, 3])
regions = np.array(["N", "S", "N", "W", "S", "W", "N"])

attention = ((latency >= 140) | (errors >= 3)) & (regions != "W")
selected = np.nonzero(attention)[0]
print(selected, latency[selected])

# Use elementwise & and |, and parenthesize every comparison.
assert selected.tolist() == [4, 6]
```

3.2 Fancy indexing gathers and assigns

```
priority = np.array([6, 3, 1])
investigation = np.column_stack([
    priority,
    latency[priority],
    errors[priority],
])

scores = np.zeros(7, dtype=int)
np.add.at(scores, [1, 1, 4, 6, 6, 6], 1)
assert scores.tolist() == [0, 2, 0, 0, 1, 0, 3]
```

Fancy indexing returns a copy. Repeated assignment such as `x[idx] += 1` may update a repeated position only once because buffered results are written back. Use `np.add.at` when repeats matter.

3.3 Sort, rank, and structure records

```
order = np.argsort(latency) # ascending positions
top3 = np.argpartition(latency, -3)[-3:]
top3 = top3[np.argsort(latency[top3])[:-1]]

events = np.zeros(7, dtype=[
    ("region", "U1"),
    ("latency_ms", "i4"),
    ("errors", "i2"),
])
events["region"] = regions
events["latency_ms"] = latency
events["errors"] = errors

ranked = np.sort(events, order=["errors", "latency_ms"])[::-1]
print(ranked[:3])
```

TRY IT YOURSELF 3.1 - Conditional selection

From arrays for price, units, and category, select rows where revenue exceeds 1,000 and category is A or B. Return both indices and values.

Deliverable: A boolean mask, integer positions, and a table-like 2D result.

Hint: Compute revenue first; use `np.isin` for categories.

TRY IT YOURSELF 3.2 - Top-k without a full sort

Generate 1,000 scores and find the indices of the ten largest using `argpartition`. Sort only those ten descending.

Deliverable: Ten indices ordered from highest to lowest, verified against a full argsort.

Hint: Partition at -10, then sort the subset.

TRY IT YOURSELF 3.3 - Challenge: incident triage

Create 30 synthetic incidents with fields `id`, `service`, `severity`, `latency_ms`, and `resolved`. Filter unresolved severity ≥ 3 incidents, reduce latency outliers by clipping at the 95th percentile, and rank by severity then latency.

Deliverable: A structured array of prioritized incidents plus counts by service.

Hint: Build one mask; copy the filtered records before mutation; use `np.unique(..., return_counts=True)`.

COMMON PITFALLS

- Use `&` and `|` for arrays, not `and/or`. Parentheses are required because comparison precedence is subtle.
- Fancy indexing copies data; basic slicing usually views it. Mutation behavior differs.
- `np.partition` does not fully sort; it only guarantees partition placement.
- Structured arrays are useful for compact low-level records; Pandas is usually clearer for analysis.

NumPy review lab

Set a 55-minute timer. Work from a clean kernel and do not consult the answer key until you have assertions for all four checkpoints.

Checkpoint	Task	Evidence
1. Model	Create a (7 days, 4 sites, 24 hours) load cube.	shape and dtype assertions
2. Compute	Apply site multipliers and calculate hourly totals.	broadcasting comment
3. Select	Find site-hours above each site's 95th percentile.	mask + positions
4. Rank	Report five largest loads with day/site/hour coordinates.	unravel_index + sorted values

```
rng = np.random.default_rng(11)
load = rng.gamma(shape=6, scale=8, size=(7, 4, 24))
site_factor = np.array([0.95, 1.00, 1.08, 1.15])

# Continue here. Before coding, write the expected shape of:
# load * site_factor[None, :, None]
```

EXIT TICKET

Explain in one sentence each: view vs copy; axis=0 vs axis=1; basic vs fancy indexing; partition vs sort; NumPy vs Pandas.

Day 2: Pandas and data analysis

Pandas adds labels, mixed dtypes, missing-value behavior, relational operations, and grouped analysis. The governing rule is alignment: values follow labels unless you explicitly ask for positional behavior.

DAY 2 CHECKPOINT

By the end, you can state the grain of every table, validate a merge, and produce a grouped result whose denominator and missing-value policy are explicit.

Session 4: Pandas Fundamentals

DAY 2 LAB | 2-3 hours

Focus: Series, DataFrame, indexing, selection, loading, inspection

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

4.1 Load a self-contained dataset

```
from io import StringIO
import pandas as pd

raw = """ticket_id,team,channel,minutes,satisfaction,resolved
T101,Core,email,35,4.6,True
T102,Edge,chat,18,4.9,True
T103,Core,web,72,,False
T104,Data,email,51,3.8,True
T105,Edge,web,44,4.1,False
T106,Data,chat,29,4.7,True
"""

tickets = pd.read_csv(StringIO(raw))
print(tickets.shape)
print(tickets.dtypes)
print(tickets.isna().sum())
assert tickets["ticket_id"].is_unique
```

A Series has values plus an index. A DataFrame is a set of aligned Series sharing an index. Before analysis, document the **grain**: one row here is one support ticket.

4.2 Select by label or position

```
tickets = tickets.set_index("ticket_id")

# Label-based; the label slice includes both endpoints.
core_cols = tickets.loc[tickets["team"].eq("Core"),
                        ["channel", "minutes", "resolved"]]

# Position-based; the stop is excluded as in normal Python slicing.
preview = tickets.iloc[:3, :4]

# assign with .loc to make target rows and columns explicit
median_score = tickets["satisfaction"].median()
tickets.loc[:, "satisfaction"] = tickets["satisfaction"].fillna(median_score)
assert tickets["satisfaction"].notna().all()
```

4.3 Inspect before transforming

```

tickets.info()
print(tickets.describe(include="all").T)
print(tickets.nunique(dropna=False))

valid_minutes = tickets["minutes"].between(0, 8 * 60)
valid_score = tickets["satisfaction"].between(1, 5)
assert valid_minutes.all() and valid_score.all()

tickets = tickets.assign(
    hours=tickets["minutes"] / 60,
    fast=tickets["minutes"].lt(40),
)

```

TRY IT YOURSELF 4.1 - Series alignment

Create two Series of inventory with overlapping but differently ordered product labels. Add them once normally and once with `add(fill_value=0)`.

Deliverable: Two results and a short explanation of missing labels.

Hint: Pandas aligns by label, not by visible row position.

TRY IT YOURSELF 4.2 - Selection contract

Select unresolved tickets with minutes above the median, returning only team, channel, and minutes. Sort descending by minutes.

Deliverable: A DataFrame and an assertion that every returned row meets both conditions.

Hint: Build the mask first, then use `loc`.

TRY IT YOURSELF 4.3 - Challenge: analyze tickets

Extend the CSV to at least 12 tickets with one duplicate id, one impossible satisfaction score, and two missing values. Build an audit, correct or exclude invalid rows, and report team-level medians.

Deliverable: A cleaned DataFrame, a three-row audit summary, and a team result.

Hint: Preserve the raw frame; derive a clean copy with explicit rules.

COMMON PITFALLS

- `loc` is label-based and its slice stop is inclusive; `iloc` is positional and its stop is exclusive.
- Chained indexing can create an ambiguous view/copy. Use one `.loc` assignment.
- `describe` is not validation. Add uniqueness, range, null, and row-count assertions.
- Automatic alignment is powerful but can introduce NaN when labels do not overlap.

Session 5: Pandas Advanced Operations

DAY 2 LAB | 2-3 hours

Focus: MultiIndex, concat, merge, join, relational validation

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

5.1 MultiIndex represents multiple label dimensions

```
idx = pd.MultiIndex.from_product(
    [ ["North", "South"], ["Basic", "Pro"] ],
    names=["region", "plan"],
)
quarterly = pd.DataFrame(
    {"q1": [120, 85, 110, 92], "q2": [128, 91, 116, 99]},
    index=idx,
)

north = quarterly.loc["North"]
by_quarter = quarterly.stack(future_stack=True).rename("accounts")
by_quarter.index.names = ["region", "plan", "quarter"]
wide_again = by_quarter.unstack("quarter")
pd.testing.assert_frame_equal(quarterly, wide_again)
```

Sort a MultiIndex before range slicing. Use named levels; unnamed levels make later code harder to read and easier to reorder incorrectly.

5.2 Concatenate batches with provenance

```
jan = pd.DataFrame({"id": [1, 2], "value": [10, 12]})
feb = pd.DataFrame({"id": [3, 4], "value": [9, 15]})

all_months = pd.concat(
    {"2025-01": jan, "2025-02": feb},
    names=["month", "row"],
)
assert len(all_months) == len(jan) + len(feb)

# DataFrame.append was removed in Pandas 2.0; use pd.concat.
flat = pd.concat([jan, feb], ignore_index=True, verify_integrity=False)
```

5.3 Merge with a cardinality contract

```
orders = pd.DataFrame({
    "order_id": [11, 12, 13, 14],
    "customer_id": [1, 2, 1, 4],
    "amount": [80, 55, 120, 40],
})
customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "segment": ["Growth", "Core", "Core"],
})

enriched = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
print(enriched["_merge"].value_counts())
assert len(enriched) == len(orders)
assert enriched["order_id"].is_unique
```

JOIN AUDIT

Before merging, state: left grain, right grain, key, expected cardinality, desired unmatched behavior, and expected row count. After merging, test each statement.

TRY IT YOURSELF 5.1 - MultiIndex slicing

Create monthly revenue indexed by region and product. Select one region, unstack product to columns, and calculate each region's share of total revenue.

Deliverable: A tidy Series, a wide table, and a region-share Series summing to 1.

Hint: Name both levels and sort the index.

TRY IT YOURSELF 5.2 - Concatenation contract

Concatenate three monthly frames whose columns differ slightly. Preserve the month as an outer key and compare outer vs inner column joins.

Deliverable: Two combined frames and a note identifying which data an inner join discards.

Hint: Use `keys=` and `join=`.

TRY IT YOURSELF 5.3 - Challenge: relational assembly

Combine customers, orders, order_items, and products into one item-level table. Use `validate` on every merge, preserve unmatched orders, and compute item revenue.

Deliverable: A documented item-level grain, a merge audit, and revenue by customer segment.

Hint: Expected relationships are many-to-one from items to orders/products and orders to customers.

COMMON PITFALLS

- Concatenation stacks objects; merge matches keys. Decide from the data relationship, not syntax convenience.
- Duplicate keys can multiply rows. `validate=` turns a silent explosion into an explicit error.
- An inner join silently drops unmatched rows. Use `left + indicator` during investigation.
- `DataFrame.append` is removed from current Pandas. Translate old examples to `pd.concat`.

Session 6: Pandas Grouping and Aggregation

DAY 2 LAB | 2-3 hours

Focus: split-apply-combine, agg, transform, filter, pivot tables

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

6.1 One dataset. three result shapes

```
rng = np.random.default_rng(9)
n = 120
ops = pd.DataFrame({
    "region": rng.choice(["North", "South", "West"], n),
    "team": rng.choice(["Core", "Edge", "Data"], n),
    "channel": rng.choice(["email", "chat", "web"], n),
    "minutes": rng.gamma(4, 10, n).round(1),
    "satisfaction": rng.normal(4.2, 0.45, n).clip(1, 5),
    "resolved": rng.random(n) > 0.18,
})

summary = ops.groupby("region", observed=True).agg(
    tickets=("minutes", "size"),
    median_minutes=("minutes", "median"),
    resolution_rate=("resolved", "mean"),
).sort_values("resolution_rate")

ops["region_minutes_z"] = ops.groupby("region")["minutes"].transform(
    lambda s: (s - s.mean()) / s.std(ddof=0)
)

large_teams = ops.groupby("team").filter(lambda g: len(g) >= 35)
```

agg reduces each group, **transform** returns one value per original row, and **filter** keeps or removes entire groups. Reserve flexible **apply** for cases that cannot be expressed more directly; it is often slower and can create surprising indices.

6.2 Pivot tables are labeled multidimensional summaries

```
pivot = pd.pivot_table(
    ops,
    index="region",
    columns="channel",
    values="minutes",
    aggfunc=["median", "count"],
    fill_value=0,
    margins=True,
)

resolution = pd.crosstab(
    ops["region"],
    ops["resolved"],
    normalize="index",
)
assert np.allclose(resolution.sum(axis=1), 1)
```

6.3 Make denominator choices visible

```
# Ticket-weighted overall rate
ticket_rate = ops["resolved"].mean()

# Equal weight per region (a different question)
region_equal_rate = ops.groupby("region")["resolved"].mean().mean()

print({"ticket_weighted": ticket_rate,
      "region_equal": region_equal_rate})
```

TRY IT YOURSELF 6.1 - Named aggregation

Group operations data by team and channel. Report row count, median minutes, 90th percentile minutes, and resolution rate with flat output column names.

Deliverable: A sorted DataFrame with one row per team-channel group.

Hint: Use named aggregation and a lambda for quantile(0.9).

TRY IT YOURSELF 6.2 - Transform for anomaly flags

Within each region, flag tickets whose minutes exceed the regional median plus two median absolute deviations.

Deliverable: A boolean column aligned to all original rows and a count by region.

Hint: Use transform twice or define a transform function.

TRY IT YOURSELF 6.3 - Challenge: decision report

Build a region x channel pivot with ticket count, median time, and resolution rate. Identify the weakest cell only among cells with at least 8 tickets, then list the five slowest tickets from that cell.

Deliverable: A pivot table, one selected cell, and a five-row evidence table.

Hint: Create grouped long-form metrics first; filter eligibility before ranking.

COMMON PITFALLS

- size counts rows; count excludes missing values in the selected column.
- A mean of group means weights groups equally, not observations equally.
- Grouping keys may become an index. Choose `as_index=False` or `reset_index` deliberately.
- pivot fails on duplicate combinations; `pivot_table` aggregates them.

Day 3: Visualization

A plot is an argument encoded with position, length, color, shape, and annotation. Start with the question, choose the minimum effective marks, and verify that the visual remains truthful when printed or viewed without the notebook context.

DAY 3 CHECKPOINT

By the end, you can build with Matplotlib's object-oriented interface, control scales and guides, and assemble a coherent dashboard rather than a collage of charts.

Session 7: Matplotlib Fundamentals

DAY 3 LAB | 2-3 hours

Focus: line, scatter, density, contour, saving figures

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

7.1 Prefer the object-oriented interface

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 200)
y = np.sin(x) * np.exp(-0.08 * x)

fig, ax = plt.subplots(figsize=(7, 3.8), constrained_layout=True)
ax.plot(x, y, color="#247BA0", linewidth=2, label="signal")
ax.axhline(0, color="0.45", linewidth=0.8)
ax.set(title="Damped operational signal", xlabel="time", ylabel="level")
ax.legend(frameon=False)
fig.savefig("signal.png", dpi=180, bbox_inches="tight")
plt.show()
```

The Figure is the canvas; each Axes is a plotting area. Keep references to both. Call `savefig` before closing the figure and choose a resolution suitable for the destination.

7.2 Scatter plots encode relationships

```
rng = np.random.default_rng(4)
n = 180
minutes = rng.gamma(4, 10, n)
satisfaction = (5.0 - 0.018 * minutes + rng.normal(0, 0.35, n)).clip(1, 5)
volume = rng.integers(10, 100, n)
priority = rng.integers(1, 5, n)

fig, ax = plt.subplots(figsize=(6.8, 4.2), constrained_layout=True)
points = ax.scatter(minutes, satisfaction, c=priority, s=volume,
                    cmap="viridis", alpha=0.7, edgecolor="white", linewidth=0.3)
ax.set(xlabel="resolution time (min)", ylabel="satisfaction",
      title="Longer resolution tends to lower satisfaction")
fig.colorbar(points, ax=ax, label="priority")
```

7.3 Contours and density summarize crowded structure

```
x = np.linspace(-3, 3, 160)
y = np.linspace(-2.5, 2.5, 140)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2)) - 0.55 * np.exp(-((X - 1.3)**2 + (Y + 0.6)**2))

fig, ax = plt.subplots(figsize=(6.4, 4.2), constrained_layout=True)
filled = ax.contourf(X, Y, Z, levels=18, cmap="coolwarm")
lines = ax.contour(X, Y, Z, levels=8, colors="black", linewidths=0.45)
ax.clabel(lines, fontsize=7)
fig.colorbar(filled, ax=ax, label="response")
ax.set(title="Response surface", xlabel="feature 1", ylabel="feature 2")
```

TRY IT YOURSELF 7.1 - Line story

Create 30 days of actual and target values. Plot both with distinct line styles, shade the gap, and label only the largest shortfall.

Deliverable: A readable figure with title, units, legend, and one decision-relevant annotation.

Hint: Use `fill_between` and `argmax` on `target - actual`.

TRY IT YOURSELF 7.2 - Scatter encoding audit

Plot response time vs satisfaction. Encode channel by color and volume by size; then write which encoding is quantitative and which is categorical.

Deliverable: A figure and a two-sentence encoding audit.

Hint: For a few categories, plot each group separately to get a categorical legend.

TRY IT YOURSELF 7.3 - Challenge: four-view figure

Build a 2x2 figure containing a line plot, scatter plot, histogram, and contour or hexbin plot from one synthetic dataset.

Deliverable: One figure with a shared title, consistent palette, units, and no overlapping labels.

Hint: Use `plt.subplots` and `constrained_layout=True`.

COMMON PITFALLS

- `plt.plot` connects points in input order. Sort `x` when the line represents a progression.
- Area and marker size can exaggerate differences. Scale sizes intentionally and provide a guide.
- Rainbow colormaps introduce false boundaries. Prefer perceptually uniform sequential maps.
- Always label units and save from the Figure object.

Session 8: Matplotlib Customization

DAY 3 LAB | 2-3 hours

Focus: legends, colorbars, subplots, annotations, ticks, styles

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

8.1 Build layout first

```
from matplotlib.ticker import PercentFormatter, MaxNLocator

fig = plt.figure(figsize=(8, 5.2), constrained_layout=True)
grid = fig.add_gridspec(2, 3)
ax_main = fig.add_subplot(grid[:, :2])
ax_top = fig.add_subplot(grid[0, 2])
ax_bottom = fig.add_subplot(grid[1, 2])

fig.suptitle("Service health overview", fontsize=14, fontweight="bold")
for ax in (ax_main, ax_top, ax_bottom):
    ax.spines[["top", "right"]].set_visible(False)
```

8.2 Legends and colorbars explain encodings

```
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable

norm = Normalize(vmin=1, vmax=5)
cmap = plt.colormaps["viridis"]
for team, marker in {"Core": "o", "Edge": "s", "Data": "^"}.items():
    sub = ops.loc[ops["team"].eq(team)]
    ax_main.scatter(sub["minutes"], sub["satisfaction"],
                    c=sub["satisfaction"], cmap=cmap, norm=norm,
                    marker=marker, alpha=0.65, label=team)

ax_main.legend(title="team", frameon=False)
fig.colorbar(ScalarMappable(norm=norm, cmap=cmap), ax=ax_main,
             label="satisfaction")
```

Use legends for discrete categories and colorbars for continuous values. If color encodes a binary state, a two-item legend is usually clearer than a continuous colorbar.

8.3 Annotations, ticks, and scoped styles

```
with plt.rc_context({
    "axes.titlesize": 11,
    "axes.labelsize": 9,
    "font.size": 9,
    "grid.alpha": 0.25,
}):
    fig, ax = plt.subplots(figsize=(6.8, 3.8), constrained_layout=True)
    rates = ops.groupby("region")["resolved"].mean().sort_values()
    bars = ax.barh(rates.index, rates.values, color="#20A39E")
    ax.xaxis.set_major_formatter(PercentFormatter(1.0))
    ax.xaxis.set_major_locator(MaxNLocator(5))
    ax.bar_label(bars, labels=[f"{v:.1%}" for v in rates], padding=3)
    ax.set(xlabel="resolution rate", title="Resolution rate by region")
    ax.set_xlim(0, 1)
```

TRY IT YOURSELF 8.1 - Honest legend

Create two lines plus an alert threshold and add a legend that includes only the two data series. Place it outside the plotting area.

Deliverable: A figure with uncluttered data and a deliberate legend.

Hint: Pass selected handles/labels or prefix the threshold label with an underscore.

TRY IT YOURSELF 8.2 - Tick formatter

Plot monthly revenue as currency and growth as percent on aligned subplots. Limit tick count and format values without changing the underlying data.

Deliverable: Two subplots with readable, semantic ticks.

Hint: Use `StrMethodFormatter` or `FuncFormatter` and `PercentFormatter`.

TRY IT YOURSELF 8.3 - Challenge: annotated publication figure

Create a two-panel figure: left shows a trend with uncertainty; right shows regional rates. Add one annotation, a coherent palette, accessible labels, and a concise source/note footer.

Deliverable: A 180-dpi PNG that remains readable at 7 inches wide.

Hint: Use `fig.text` for a footer and `rc_context` so style changes stay local.

COMMON PITFALLS

- Legends should decode marks, not repeat every annotation or reference line.
- A colorbar must share the exact `Normalize` and `colormap` used by the plotted artist.
- Global `rcParams` changes leak into later cells. Prefer `rc_context` for reproducibility.
- Excess tick precision suggests false accuracy and creates visual noise.

Session 9: Advanced Visualization

DAY 3 LAB | 2-3 hours

Focus: 3D plotting, Seaborn, faceting, professional dashboards

Rhythm: 20 min concept map; 45-60 min code-along; 45 min practice; 30-60 min challenge and review.

9.1 Use 3D when geometry is the question

```
x = np.linspace(-2.5, 2.5, 80)
y = np.linspace(-2.5, 2.5, 80)
X, Y = np.meshgrid(x, y)
Z = np.sin(X**2 + Y**2) / (1 + X**2 + Y**2)

fig = plt.figure(figsize=(7, 5), constrained_layout=True)
ax = fig.add_subplot(111, projection="3d")
surface = ax.plot_surface(X, Y, Z, cmap="viridis", linewidth=0,
                          antialiased=True)
ax.set(xlabel="x", ylabel="y", zlabel="response", title="Response surface")
fig.colorbar(surface, ax=ax, shrink=0.65, pad=0.08)
```

3D adds occlusion and perspective distortion. Use it for surfaces, trajectories, or spatial geometry; prefer contours, heatmaps, or small multiples for precise comparison.

9.2 Seaborn maps tidy columns to semantics

```
sns.set_theme(style="ticks", context="notebook")

g = sns.relplot(
    data=ops,
    x="minutes",
    y="satisfaction",
    hue="team",
    col="region",
    size="resolved",
    sizes=(24, 70),
    alpha=0.7,
    height=3.0,
    aspect=0.85,
)
g.set_axis_labels("resolution time (min)", "satisfaction")
g.set_titles("{col_name}")
g.figure.suptitle("Time and satisfaction by region", y=1.04)
```

9.3 Dashboard: one question per panel

A useful dashboard has a reading order: headline metric, comparison, distribution, and diagnostic detail. Reuse category colors, align related axes, and remove panels that do not change a decision.

```
fig, axes = plt.subplots(2, 2, figsize=(9, 6), constrained_layout=True)
fig.suptitle("Support operations: speed, quality, and volume", fontweight="bold")

sns.barplot(data=ops, x="region", y="resolved", errorbar=("ci", 90), ax=axes[0, 0])
sns.boxplot(data=ops, x="team", y="minutes", showliers=False, ax=axes[0, 1])
sns.histplot(data=ops, x="minutes", hue="resolved", element="step",
             stat="density", common_norm=False, ax=axes[1, 0])
sns.scatterplot(data=ops, x="minutes", y="satisfaction", hue="team",
               alpha=0.65, ax=axes[1, 1])

axes[0, 0].set(ylabel="resolution rate", title="Outcome")
axes[0, 1].set(title="Speed distribution")
axes[1, 0].set(title="Resolved vs unresolved")
axes[1, 1].set(title="Speed-quality relationship")
sns.despine(fig=fig)
```

TRY IT YOURSELF 9.1 - 3D vs 2D

Plot the same mathematical surface as a 3D surface and a filled contour. List two tasks for which each view is better.

Deliverable: Two figures plus a four-point comparison.

Hint: Use identical data, colormap, and color limits.

TRY IT YOURSELF 9.2 - Faceted distribution

Compare resolution-time distributions by team across regions with a Seaborn figure-level function.

Deliverable: Facets with shared bins/scales and a concise observation.

Hint: Use `displot` with `col=` and `hue=`, or a `FacetGrid`.

TRY IT YOURSELF 9.3 - Challenge: complete dashboard

Build a four-panel decision dashboard for operations leaders. Include outcome, speed, volume, and diagnostic relationship; highlight one actionable segment.

Deliverable: A coherent figure, a 100-word narrative, and three accessibility checks.

Hint: Use a limited palette, direct titles, and one annotation tied to the narrative.

COMMON PITFALLS

- 3D can obscure values. Always ask whether a 2D encoding answers the question more precisely.
- Figure-level Seaborn functions own their Figure; customize through the returned grid object.
- Too many hue/size/style encodings overwhelm working memory. Use only decision-relevant semantics.
- A dashboard is not a chart collection. Give it a reading order and a unifying question.

Final capstone: Support Operations Intelligence

Scenario: You support a growing AI product. Leaders want to know where support outcomes are weakening and which operational segment deserves intervention. Build a restartable analysis from raw synthetic records to a decision-ready figure.

DECISION QUESTION

Which region-channel combination has the weakest resolution outcome after requiring enough evidence, and what relationship between workload, response time, and satisfaction might explain it?

Deliverables and rubric

Dimension	Excellent evidence	Points
Reproducibility	Seeded generator; Restart and Run All succeeds	20
Data quality	Grain, key, missing, range, and category checks	20
Analysis	Vectorized features; validated joins; defensible groups	25
Visualization	Accurate encodings, units, hierarchy, annotation	25
Narrative	One finding, caveat, and next action	10

Step 1 - Generate raw tables

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 900
teams = pd.DataFrame({
    "team_id": ["TM1", "TM2", "TM3", "TM4"],
    "team": ["Core", "Edge", "Data", "Safety"],
    "capacity": [1.00, 0.88, 1.12, 0.94],
})

raw = pd.DataFrame({
    "ticket_id": [f"TK{i:04d}" for i in range(n)],
    "opened_at": pd.Timestamp("2025-01-01") +
        pd.to_timedelta(rng.integers(0, 90 * 24, n), unit="h"),
    "team_id": rng.choice(teams["team_id"], n, p=[.35, .25, .25, .15]),
    "region": rng.choice(["North", "South", "West"], n, p=[.38, .34, .28]),
    "channel": rng.choice(["email", "chat", "web"], n, p=[.34, .41, .25]),
    "priority": rng.integers(1, 5, n),
    "message_tokens": rng.gamma(5, 90, n).round(),
})

base = rng.gamma(4.5, 9.0, n)
region_penalty = raw["region"].map({"North": 0, "South": 8, "West": 15})
channel_penalty = raw["channel"].map({"chat": 0, "email": 8, "web": 4})
raw["minutes"] = (base + region_penalty + channel_penalty +
    3 * raw["priority"] + rng.normal(0, 5, n)).clip(2)
prob = 1 / (1 + np.exp((raw["minutes"] - 62) / 16))
raw["resolved"] = rng.random(n) < prob
raw["satisfaction"] = (4.9 - .018 * raw["minutes"] +
    .20 * raw["resolved"] + rng.normal(0, .35, n)).clip(1, 5)

# Inject realistic quality defects.
raw.loc[rng.choice(n, 18, replace=False), "satisfaction"] = np.nan
raw.loc[rng.choice(n, 5, replace=False), "minutes"] = -1
raw = pd.concat([raw, raw.iloc[[20]], ignore_index=True)
```

Step 2 - Audit and clean

```
audit = pd.Series({
    "rows": len(raw),
    "duplicate_ids": raw["ticket_id"].duplicated().sum(),
    "missing_satisfaction": raw["satisfaction"].isna().sum(),
    "invalid_minutes": (~raw["minutes"].between(0, 480)).sum(),
    "unknown_team_ids": (~raw["team_id"].isin(teams["team_id"])).sum(),
})
print(audit)

clean = raw.drop_duplicates("ticket_id", keep="first").copy()
clean = clean.loc[clean["minutes"].between(0, 480)].copy()
clean["satisfaction"] = clean["satisfaction"].fillna(
    clean.groupby(["region", "channel"])["satisfaction"].transform("median")
)
clean["opened_at"] = pd.to_datetime(clean["opened_at"], errors="raise")

assert clean["ticket_id"].is_unique
assert clean["minutes"].between(0, 480).all()
assert clean["satisfaction"].between(1, 5).all()
```

Step 3 - Enrich and engineer features

```
df = clean.merge(
    teams,
    on="team_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
assert df["_merge"].eq("both").all()
df = df.drop(columns="_merge")

minutes = df["minutes"].to_numpy()
priority = df["priority"].to_numpy()
capacity = df["capacity"].to_numpy()

df["load_score"] = np.log1p(df["message_tokens"].to_numpy()) * priority / capacity
df["sla_minutes"] = np.select(
    [priority == 4, priority == 3],
    [35.0, 50.0],
    default=75.0,
)
df["sla_met"] = minutes <= df["sla_minutes"].to_numpy()
df["month"] = df["opened_at"].dt.to_period("M").astype(str)
```

Step 4 - Analyze eligible segments

```
segments = df.groupby(["region", "channel", as_index=False]).agg(
    tickets=("ticket_id", "size"),
    resolution_rate=("resolved", "mean"),
    sla_rate=("sla_met", "mean"),
    median_minutes=("minutes", "median"),
    p90_minutes=("minutes", lambda s: s.quantile(.90)),
    mean_satisfaction=("satisfaction", "mean"),
)

eligible = segments.loc[segments["tickets"].ge(50)].copy()
weakest = eligible.sort_values(
    ["resolution_rate", "sla_rate", "tickets"],
    ascending=[True, True, False],
).iloc[0]
print(weakest)

focus = df.loc[
    df["region"].eq(weakest["region"]) &
    df["channel"].eq(weakest["channel"])
].copy()
```

Step 5 - Build the decision figure

```
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import PercentFormatter

sns.set_theme(style="whitegrid", context="notebook")
fig, axes = plt.subplots(2, 2, figsize=(10, 7), constrained_layout=True)
fig.suptitle("Support Operations Intelligence", fontsize=16, fontweight="bold")

order = segments.groupby("region")["resolution_rate"].mean().sort_values().index
sns.barplot(data=segments, x="region", y="resolution_rate", order=order,
            color="#247BA0", errorbar=None, ax=axes[0, 0])
axes[0, 0].yaxis.set_major_formatter(PercentFormatter(1))
axes[0, 0].set(title="Resolution outcome", xlabel="", ylabel="resolution rate",
              ylim=(0, 1))

sns.boxplot(data=df, x="channel", y="minutes", hue="region",
            showfliers=False, ax=axes[0, 1])
axes[0, 1].set(title="Time distribution", xlabel="", ylabel="minutes")
axes[0, 1].legend(title="region", frameon=False, fontsize=8)

trend = df.groupby(["month", "region"], as_index=False)["sla_met"].mean()
sns.lineplot(data=trend, x="month", y="sla_met", hue="region",
            marker="o", ax=axes[1, 0])
axes[1, 0].yaxis.set_major_formatter(PercentFormatter(1))
axes[1, 0].set(title="SLA trend", xlabel="month", ylabel="SLA met")
axes[1, 0].legend(title="region", frameon=False, fontsize=8)

sns.scatterplot(data=focus, x="load_score", y="satisfaction",
                hue="resolved", alpha=.65, ax=axes[1, 1])
axes[1, 1].set(title=f"Weakest segment: {weakest['region']} / {weakest['channel']}",
              xlabel="load score", ylabel="satisfaction")

sns.despine(fig=fig)
fig.text(.01, .01, "Synthetic training data; eligibility: at least 50 tickets.",
        fontsize=8, color="0.35")
fig.savefig("support_operations_capstone.png", dpi=180, bbox_inches="tight")
```

Step 6 - Write the decision narrative

NARRATIVE TEMPLATE

Finding: Among segments with at least 50 tickets, [segment] has the weakest resolution rate at [value]. **Evidence:** It also shows [supporting metric/pattern]. **Caveat:** These observational, synthetic records do not establish causality. **Action:** Inspect staffing, routing, and ticket mix for that segment; track the same metrics after a controlled operational change.

Capstone extensions

- Core: add an audit table to the notebook and export segments to CSV.
- Mastery: calculate bootstrap confidence intervals for segment resolution rates with NumPy.
- AI bridge: turn the final metrics and caveats into a small JSON object suitable for a RAG document or agent tool response.
- Engineering bridge: move generation, cleaning, and metrics into tested functions; leave the notebook as orchestration and narrative.

COMMON PITFALLS

- Cleaning decisions must be visible. Never silently drop a bad row without an audit count.
- A weakest small segment may be noise. Set eligibility before ranking, not after seeing results.
- A relationship in a scatter plot does not identify a cause. State plausible confounders.
- A capstone is complete only after Restart and Run All and artifact verification.

Answer key

Use these as reference solutions, not canonical answers. Equivalent code is correct when it preserves the stated grain/shape, handles edge cases, and passes meaningful assertions. Variables such as ops refer to the dataset created in the relevant session.

Session 1

Exercise 1.1

```
a = np.arange(1, 25).reshape(2, 3, 4)
result = a[1, 0, -2:]
assert result.shape == (2,)
np.testing.assert_array_equal(result, [15, 16])
```

Why it works: The integer indices remove day and site axes; the final slice preserves a length-2 hour axis.

Exercise 1.2

```
sites = [np.arange(i, i + 4) for i in (0, 10, 20)]
matrix = np.vstack(sites)
fifth = np.array([4, 14, 24])[:, None]
matrix = np.hstack([matrix, fifth])
parts = np.split(matrix, [1, 2], axis=0)
assert matrix.shape == (3, 5)
assert all(part.shape == (1, 5) for part in parts)
```

Why it works: Turn the fifth-hour vector into a column before horizontal concatenation.

Exercise 1.3

```
cube = np.arange(100, 124).reshape(2, 3, 4)
daily = cube.sum(axis=(1, 2))
by_store = cube.sum(axis=(0, 2))
chosen = cube[..., [1, 3]].copy()
swapped = cube.swapaxes(0, 1)
assert daily.shape == (2,)
assert by_store.shape == (3,)
assert chosen.shape == (2, 3, 2)
assert swapped.shape == (3, 2, 4)
```

Why it works: Tuple axes aggregate more than one dimension. Fancy selection of products returns a copy.

Session 2

Exercise 2.1

```
rng = np.random.default_rng(5)
x = rng.normal(size=(100, 4))
z = (x - x.mean(0, keepdims=True)) / x.std(0, keepdims=True)
np.testing.assert_allclose(z.mean(0), 0, atol=1e-12)
np.testing.assert_allclose(z.std(0), 1, atol=1e-12)
```

Why it works: keepdims preserves shape (1, 4), which broadcasts down the 100 rows.

Exercise 2.2

```
points = np.array([[0, 0], [1, 0], [0, 2], [2, 2], [-1, 1]])
delta = points[:, None, :] - points[None, :, :]
dist = np.sqrt((delta ** 2).sum(axis=2))
np.testing.assert_allclose(dist, dist.T)
np.testing.assert_allclose(np.diag(dist), 0)
```

Why it works: The two inserted axes create every point-pair combination; the last axis holds coordinates.

Exercise 2.3

```
def tiered_bill(usage):
    usage = np.asarray(usage, dtype=float)
    tier1 = np.minimum(usage, 100) * .12
    tier2 = np.minimum(np.clip(usage - 100, 0, None), 200) * .18
    tier3 = np.clip(usage - 300, 0, None) * .25
    return tier1 + tier2 + tier3

u = np.array([0, 100, 300, 450])
np.testing.assert_allclose(tiered_bill(u), [0, 12, 48, 85.5])
```

Why it works: Each tier is calculated independently. Vectorization removes index bookkeeping and branch logic.

Session 3

Exercise 3.1

```
price = np.array([25, 80, 15, 55, 120.])
units = np.array([50, 8, 100, 30, 12])
category = np.array(["A", "C", "B", "B", "D"])
revenue = price * units
mask = (revenue > 1000) & np.isin(category, ["A", "B"])
positions = np.flatnonzero(mask)
result = np.column_stack([price[mask], units[mask], revenue[mask]])
assert np.all(result[:, 2] > 1000)
```

Why it works: `np.isin` expresses categorical membership elementwise.

Exercise 3.2

```
rng = np.random.default_rng(8)
scores = rng.normal(size=1000)
idx = np.argpartition(scores, -10)[-10:]
idx = idx[np.argsort(scores[idx])[:-1]]
expected = np.argsort(scores)[-10:][:-1]
np.testing.assert_array_equal(idx, expected)
```

Why it works: `argpartition` reduces work when `k` is small; sort only the selected `k` values.

Exercise 3.3

```
rng = np.random.default_rng(3)
inc = np.zeros(30, dtype=[("id", "i4"), ("service", "U8"),
    ("severity", "i2"), ("latency_ms", "f8"), ("resolved", "?")])
inc["id"] = np.arange(100, 130)
inc["service"] = rng.choice(["api", "search", "auth"], 30)
inc["severity"] = rng.integers(1, 5, 30)
inc["latency_ms"] = rng.lognormal(4.8, .6, 30)
inc["resolved"] = rng.random(30) > .35
focus = inc[(~inc["resolved"]) & (inc["severity"] >= 3)].copy()
cap = np.percentile(focus["latency_ms"], 95)
focus["latency_ms"] = np.clip(focus["latency_ms"], None, cap)
ranked = np.sort(focus, order=["severity", "latency_ms"])[::-1]
services, counts = np.unique(ranked["service"], return_counts=True)
```

Why it works: Copy before clipping so the original incident log remains an audit source.

Session 4

Exercise 4.1

```
a = pd.Series({"A": 10, "B": 8, "C": 4})
b = pd.Series({"C": 7, "A": 2, "D": 5})
ordinary = a + b
complete = a.add(b, fill_value=0)
assert ordinary.isna().sum() == 2
assert complete.loc["D"] == 5
```

Why it works: Normal arithmetic uses the union of labels and yields NaN where either side is missing.

Exercise 4.2

```
mask = (~tickets["resolved"]) & tickets["minutes"].gt(tickets["minutes"].median())
result = tickets.loc[mask, ["team", "channel", "minutes"]].sort_values(
    "minutes", ascending=False)
assert ((~tickets.loc[result.index, "resolved"]) &
        result["minutes"].gt(tickets["minutes"].median())).all()
```

Why it works: Build one mask against the source frame, then make one label-based selection.

Exercise 4.3

```
audit = pd.Series({
    "rows": len(raw_tickets),
    "duplicate_ids": raw_tickets["ticket_id"].duplicated().sum(),
    "invalid_scores": (~raw_tickets["satisfaction"].between(1, 5) &
        raw_tickets["satisfaction"].notna()).sum(),
})
clean = raw_tickets.drop_duplicates("ticket_id").copy()
clean.loc[~clean["satisfaction"].between(1, 5), "satisfaction"] = np.nan
clean["satisfaction"] = clean["satisfaction"].fillna(
    clean.groupby("team")["satisfaction"].transform("median"))
team_result = clean.groupby("team", as_index=False).agg(
    tickets=("ticket_id", "size"), median_minutes=("minutes", "median"))
```

Why it works: One defensible policy is to convert impossible scores to missing, then impute within team.

Session 5

Exercise 5.1

```
idx = pd.MultiIndex.from_product(
    [["North", "South"], ["A", "B"], ["Jan", "Feb"]],
    names=["region", "product", "month"])
revenue = pd.Series(np.arange(8) + 10, index=idx, name="revenue").sort_index()
north = revenue.loc["North"]
wide = revenue.unstack("product")
share = revenue.groupby("region").sum()
share = share / share.sum()
np.testing.assert_allclose(share.sum(), 1)
```

Why it works: A named, sorted index makes selection and reshaping predictable.

Exercise 5.2

```
frames = {
    "Jan": pd.DataFrame({"id": [1], "value": [10], "note": ["ok"]}),
    "Feb": pd.DataFrame({"id": [2], "value": [11]}),
    "Mar": pd.DataFrame({"id": [3], "value": [12], "owner": ["Mina"]}),
}
outer = pd.concat(frames, names=["month", "row"], join="outer")
inner = pd.concat(frames, names=["month", "row"], join="inner")
assert set(inner.columns) == {"id", "value"}
```

Why it works: The inner column join discards note and owner because they are not common to every frame.

Exercise 5.3

```
item_fact = order_items.merge(
    products, on="product_id", how="left", validate="many_to_one", indicator=True)
assert item_fact["_merge"].eq("both").all()
item_fact = item_fact.drop(columns="_merge").merge(
    orders, on="order_id", how="left", validate="many_to_one")
item_fact = item_fact.merge(
    customers, on="customer_id", how="left", validate="many_to_one")
item_fact["item_revenue"] = item_fact["quantity"] * item_fact["unit_price"]
by_segment = item_fact.groupby("segment")["item_revenue"].sum()
```

Why it works: The result grain is one order item. Validate every dimension-table lookup as many-to-one.

Session 6

Exercise 6.1

```
result = ops.groupby(["team", "channel"], as_index=False).agg(
    tickets=("minutes", "size"),
    median_minutes=("minutes", "median"),
    p90_minutes=("minutes", lambda s: s.quantile(.90)),
    resolution_rate=("resolved", "mean"),
).sort_values(["resolution_rate", "tickets"], ascending=[True, False])
```

Why it works: Named aggregation produces flat, meaningful column names.

Exercise 6.2

```
def above_robust_limit(s):
    median = s.median()
    mad = (s - median).abs().median()
    return s > median + 2 * mad

ops["slow_outlier"] = ops.groupby("region")["minutes"].transform(
    above_robust_limit)
counts = ops.groupby("region")["slow_outlier"].sum()
```

Why it works: transform guarantees an output aligned to the original row index.

Exercise 6.3

```
long = ops.groupby(["region", "channel"], as_index=False).agg(
    tickets=("minutes", "size"), median_minutes=("minutes", "median"),
    resolution_rate=("resolved", "mean"))
eligible = long.loc[long["tickets"] >= 8]
weak = eligible.sort_values(["resolution_rate", "median_minutes"],
    ascending=[True, False]).iloc[0]
evidence = ops.loc[
    ops["region"].eq(weak["region"]) & ops["channel"].eq(weak["channel"])
].nlargest(5, "minutes")
pivot = long.pivot(index="region", columns="channel",
    values=["tickets", "median_minutes", "resolution_rate"])
```

Why it works: Select eligibility in long form, rank there, and pivot only for presentation.

Session 7

Exercise 7.1

```
days = np.arange(30)
target = 100 + .8 * days
actual = target + rng.normal(0, 7, 30)
gap = target - actual
i = np.argmax(gap)
fig, ax = plt.subplots(figsize=(7, 4), constrained_layout=True)
ax.plot(days, actual, label="actual")
ax.plot(days, target, "--", label="target")
ax.fill_between(days, actual, target, alpha=.15)
ax.annotate(f"largest shortfall: {gap[i]:.1f}", (days[i], actual[i]),
    xytext=(8, -28), textcoords="offset points", arrowprops={"arrowstyle": "->"})
ax.set(xlabel="day", ylabel="units", title="Actual vs target")
ax.legend(frameon=False)
```

Why it works: Sorting is unnecessary because days are already ordered; label the unit of the gap.

Exercise 7.2

```
fig, ax = plt.subplots(figsize=(6.5, 4), constrained_layout=True)
for name, group in ops.groupby("channel"):
    ax.scatter(group["minutes"], group["satisfaction"],
        s=20 + group.index.to_numpy() % 60, alpha=.6, label=name)
ax.set(xlabel="resolution time (min)", ylabel="satisfaction")
ax.legend(title="channel", frameon=False)
```

Why it works: Color is categorical; marker area is quantitative. In real work, the size variable needs a guide.

Exercise 7.3

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6), constrained_layout=True)
axes[0, 0].plot(np.arange(30), np.cumsum(rng.normal(size=30)))
axes[0, 1].scatter(ops["minutes"], ops["satisfaction"], alpha=.5)
axes[1, 0].hist(ops["minutes"], bins=18, color="#20A39E")
axes[1, 1].hexbin(ops["minutes"], ops["satisfaction"], gridsize=18, cmap="Blues")
for ax, title in zip(axes.flat, ["Trend", "Relationship", "Distribution", "Density"]):
    ax.set_title(title)
fig.suptitle("Four complementary views", fontweight="bold")
```

Why it works: Each panel answers a distinct question; the shared title provides hierarchy.

Session 8

Exercise 8.1

```
fig, ax = plt.subplots(figsize=(7, 3.8), constrained_layout=True)
l1, = ax.plot(x, y1, label="actual")
l2, = ax.plot(x, y2, label="forecast")
ax.axhline(limit, color="crimson", linestyle=":", label="_threshold")
ax.legend(handles=[l1, l2], loc="center left", bbox_to_anchor=(1.02, .5),
          frameon=False)
```

Why it works: Explicit handles make the legend a deliberate decoding key.

Exercise 8.2

```
from matplotlib.ticker import StrMethodFormatter, PercentFormatter, MaxNLocator
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5), constrained_layout=True)
axes[0].plot(months, revenue)
axes[0].yaxis.set_major_formatter(StrMethodFormatter("${x:,.0f}"))
axes[1].plot(months, growth)
axes[1].yaxis.set_major_formatter(PercentFormatter(1.0))
for ax in axes:
    ax.yaxis.set_major_locator(MaxNLocator(5))
```

Why it works: Formatters change labels, not data. Keep growth stored as a proportion when using PercentFormatter(1).

Exercise 8.3

```
with plt.rc_context({"font.size": 9, "grid.alpha": .2}):
    fig, axes = plt.subplots(1, 2, figsize=(7, 3.6), constrained_layout=True)
    axes[0].plot(x, mean, color="#247BA0")
    axes[0].fill_between(x, low, high, color="#247BA0", alpha=.18)
    axes[0].annotate("intervention", (x[j], mean[j]), xytext=(10, 20),
                    textcoords="offset points", arrowprops={"arrowstyle": "->"})
    axes[1].barh(regions, rates, color="#20A39E")
    axes[1].xaxis.set_major_formatter(PercentFormatter(1))
    fig.text(.01, .01, "Synthetic data; band shows 90% interval.", fontsize=7)
```

Why it works: Verify at final output size: labels readable, no clipping, and meaning does not depend on color alone.

Session 9

Exercise 9.1

```
fig = plt.figure(figsize=(9, 4), constrained_layout=True)
ax3 = fig.add_subplot(121, projection="3d")
s = ax3.plot_surface(X, Y, Z, cmap="viridis", vmin=Z.min(), vmax=Z.max())
ax2 = fig.add_subplot(122)
c = ax2.contourf(X, Y, Z, levels=18, cmap="viridis", vmin=Z.min(), vmax=Z.max())
fig.colorbar(c, ax=[ax3, ax2], shrink=.75, label="response")
```

Why it works: 3D conveys surface geometry; contour supports precise location and level comparison.

Exercise 9.2

```
g = sns.displot(data=ops, x="minutes", col="region", hue="team",
               kind="hist", bins=np.arange(0, 121, 10),
               stat="density", common_norm=False, height=3)
g.set_axis_labels("resolution time (min)", "density")
g.set_titles("{col_name}")
```

Why it works: Shared bins and axes make distributions comparable across facets.

Exercise 9.3

```
# A valid solution is the 2x2 dashboard from the Session 9 code-along.
# Add an evidence-led annotation and a short narrative. Accessibility checks:
# 1) units are present; 2) categories remain distinguishable without color;
# 3) text is readable at final size; 4) palette is color-vision friendly.
```

Why it works: The dashboard earns its place only if the four panels support one operational decision.

Common mistake clinic

Symptom	Likely cause	Diagnostic / repair
Broadcast error	Trailing dimensions incompatible	Write both shapes right-aligned; insert a size-1 axis intentionally.
Wrong aggregate	Axis meaning reversed	Name axes in a comment and inspect a tiny hand-checkable array.
Source array changed	Slice was a view	Use <code>np.shares_memory</code> ; copy at the ownership boundary.
Mask error	Used <code>and/or</code> or omitted parentheses	Use <code>(cond1) & (cond2)</code> ; print mask dtype and sum.
Unexpected NaN	Label alignment or unmatched join	Inspect index/key sets and use merge indicator.
Row explosion	Many-to-many merge	Check key duplicates; add <code>validate</code> before trusting output.
Group result misaligned	agg used where transform was needed	Compare result length with source length.
Unreadable plot	Too many encodings or no hierarchy	Remove one encoding; state one question per panel.
Notebook fails clean run	Hidden state / out-of-order cells	Restart, Run All, then move constants/imports earlier.

Quick reference

IPython and Jupyter

Need	Command	Note
Object help / source	name? / name??	Source may be unavailable for compiled objects
Discover names	Tab / *pattern*?	Explore attributes or wildcard matches
Run code	%run script.py	Definitions remain available afterward
Time code	%timeit expression	Use %%timeit for a whole cell
Inspect history	%history -n	In, Out, _, __, _N expose session history
Debug	%debug / %pdb on	Inspect the most recent traceback
Profile	%prun statement	Use after measurement identifies a slow path
Shell	!command	Use sparingly; notebook portability matters

NumPy

Intent	Functions / syntax	Watch
Create	array, zeros, ones, arange, linspace	dtype and endpoint semantics
Inspect	shape, ndim, size, dtype	write axis meanings
Reshape	reshape, ravel, newaxis, swapaxes	element count; view behavior
Combine	concatenate, stack, vstack, column_stack	existing vs new axis
Split	split, array_split	equal vs uneven sections
Compute	ufuncs; + - * / **	broadcast alignment
Aggregate	sum, mean, std, min, max, quantile	axis, dtype, keepdims
Missing	isnan, nanmean, nansum	NaN only in suitable dtypes
Select	mask, nonzero, where, isin	parentheses with & and
Gather	a[idx], take, take_along_axis	fancy indexing copies
Sort	sort, argsort, partition, argpartition	partition is partial
Validate	assert_allclose, assert_array_equal	choose numeric tolerance

Pandas

Intent	Functions / syntax	Watch
Load	read_csv, read_parquet	dtype, parse_dates, NA tokens
Inspect	head, info, describe, value_counts	grain and key uniqueness
Select	loc, iloc, query	label vs position
Missing	isna, dropna, fillna	document the policy
Sort	sort_values, sort_index, nlargest	stable tie-breakers
Reshape	set_index, reset_index, stack, unstack	named/sorted levels
Combine	concat	axis, keys, join, duplicate index
Relate	merge, join	validate, indicator, cardinality
Group	groupby, agg, transform, filter	result shape and denominator
Summarize	pivot_table, crosstab	duplicates require aggregation
Validate	assert_frame_equal; Series checks	row count, keys, ranges

Matplotlib and Seaborn

Intent	API	Design check
Start	fig, ax = plt.subplots(...)	final size and layout
Line	ax.plot, ax.fill_between	ordered x; uncertainty meaning
Scatter	ax.scatter / sns.scatterplot	alpha; size/color guides
Distribution	hist, histplot, boxplot, violinplot	bins and sample size
Density	contourf, hexbin, kdeplot	shared color scale
Layout	subplots, add_gridspec	reading order
Guide	legend, colorbar	discrete vs continuous
Explain	set, annotate, text	units; selective annotation
Ticks	Locator / Formatter classes	precision and clutter
Style	sns.set_theme, plt.rc_context	scope changes locally
Save	fig.savefig(..., dpi=180)	inspect actual output

LIBRARY CHOICE

Use **NumPy** for dense homogeneous numerical arrays and algorithms; **Pandas** for labeled tables, missing data, joins, and groups; **Matplotlib** for precise figure construction; **Seaborn** for concise statistical graphics from tidy DataFrames.

Resources and next steps

Deepen the covered material

- Book Part I, Chapters 1-3: Jupyter/IPython help, interactive features, debugging, and profiling.
- Book Part II, Chapters 4-12: dtypes, arrays, ufuncs, aggregates, broadcasting, masks, fancy indexing, sorting, and structured arrays.
- Book Part III, Chapters 13-20: Pandas objects through aggregation/grouping. The prompt also asks for pivot tables; revisit Chapter 21 for a deeper treatment.
- Book Part IV, Chapters 25-36: Matplotlib fundamentals through 3D plotting and Seaborn.

Reference: Jake VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*, 2nd edition, O'Reilly Media, 2023, ISBN 978-1-098-12122-8. This tutorial is an original companion curriculum and does not reproduce the book's examples.

Official practice resources

- [NumPy user guide](https://numpy.org/doc/stable/user/) - <https://numpy.org/doc/stable/user/>
- [Pandas user guide](https://pandas.pydata.org/docs/user_guide/) - https://pandas.pydata.org/docs/user_guide/
- [Matplotlib tutorials](https://matplotlib.org/stable/tutorials/) - <https://matplotlib.org/stable/tutorials/>
- [Seaborn tutorial](https://seaborn.pydata.org/tutorial.html) - <https://seaborn.pydata.org/tutorial.html>
- [Jupyter documentation](https://docs.jupyter.org/) - <https://docs.jupyter.org/>
- [Book notebooks](https://github.com/jakevdp/PythonDataScienceHandbook) - <https://github.com/jakevdp/PythonDataScienceHandbook>

Bridge to AI application engineering

1. Data contracts

Turn capstone cleaning rules into functions with tests. This becomes the validation layer for model inputs and tool outputs.

2. Embeddings and transformers

Learn vector representations, attention, tokenization, inference batching, and evaluation. NumPy shape fluency transfers directly.

3. Retrieval-augmented generation

Practice chunk metadata, retrieval evaluation, and grounded answer datasets. Pandas is ideal for experiment tables and error analysis.

4. Agents

Design typed tools, state, retries, and traces. Use DataFrames to analyze trajectories and Matplotlib to visualize latency, cost, and success rates.

5. Portfolio

Publish the capstone notebook, a tested data module, and a one-page decision memo. Reproducibility is more persuasive than decorative complexity.

30-DAY RETENTION PLAN

Week 1: redo one exercise per session from memory. Week 2: repeat the capstone with a new seed and one new metric. Week 3: use a public CSV and write a data contract. Week 4: package the cleaning and metrics functions with tests, then explain the analysis to another learner.

Index

Page references point to the principal explanation or exercise for each term.

3

3D 23

3D plotting 23

3D plotting, Seaborn, faceting, professional dashboards 23

A

Advanced Visualization 23

agents 36

agg 17

aggregation 8

AI learning path 36

annotation 21, 27

answer key 28

append 15

argsort 10

array creation 6

arrays 6

arrays, dtypes, indexing, slicing, reshape, concatenate, split 6

audit 26

Axes 19

axis 8

B

book chapter map 36

boolean logic 10

boolean masks 10

broadcasting 8

broadcasting rules 8

C

capstone 25

capstone solution 25

catplot 23

caveat 27

code-along 4

color encoding 19

colorbar 21

colormap 21

common mistakes 33

comparison 10

comparisons, masks, boolean logic, fancy indexing, sorting, structured arrays 10

I

installation 4

IPython 4

J

join 15

joins 15

Jupyter 4

K

keepdims 8

keys 15

L

learning objectives 4

legend 21

legends, colorbars, subplots, annotations, ticks, styles 21

line plots 19

line, scatter, density, contour, saving figures 19

loc 13

M

magic commands 34

margins 17

mask 10

Matplotlib 19

Matplotlib Customization 21

Matplotlib functions 35

Matplotlib Fundamentals 19

merge 15

merge validation 26

missing data 14

missing values 26

MultiIndex 15

MultiIndex, concat, merge, join, relational validation 15

N

NaN 8

narrative 27

ndarray 6

newaxis 8

Normalize 21

NumPy 6

concat 15
concatenate 7
constrained_layout 21
contour 19
copy 6
crosstab 17

D

dashboard 23
data cleaning 25
DataFrame 13
debugging 33
decision figure 27
denominator 17
density 19
describe 14
documentation 36
dtype 6
duplicates 26

E

eligibility threshold 26
end-to-end workflow 25
environment 4
exercise 1.1 28
exercise 1.2 28
exercise 1.3 28
exercise 2.1 28
exercise 2.2 28
exercise 2.3 29
exercise 3.1 29
exercise 3.2 29
exercise 3.3 29
exercise 4.1 29
exercise 4.2 30
exercise 4.3 30
exercise 5.1 30
exercise 5.2 30
exercise 5.3 30
exercise 6.1 31
exercise 6.2 31
exercise 6.3 31
exercise 7.1 31
exercise 7.2 31

NumPy Advanced Indexing 10
NumPy Advanced Operations 8
NumPy functions 34
NumPy Fundamentals 6
NumPy review 12

O

object-oriented API 19

P

pairplot 23
Pandas 13
Pandas Advanced Operations 15
Pandas functions 35
Pandas Fundamentals 13
Pandas Grouping and Aggregation 17
partition 10
performance 8
pivot_table 17
plot customization 21
profiling 34
put 10

Q

quick reference 34

R

RAG 36
random number generator 4
rcParams 21
read_csv 13
relational algebra 15
relplot 23
reproducibility 4
reshape 7
resources 36
retrieval practice 4
rubric 25

S

savefig 19
scatter 19
schedule 5
Seaborn 19, 23
Seaborn functions 35
Series 13
Series, DataFrame, indexing, selection, loading, inspection 13

exercise 7.3 32

exercise 8.1 32

exercise 8.2 32

exercise 8.3 32

exercise 9.1 32

exercise 9.2 32

exercise 9.3 33

extensions 27

F

FacetGrid 23

fancy indexing 10

feature engineering 26

Figure 19

filter 17

G

GridSpec 21

groupby 17

GroupBy 17

grouped analysis 26

H

hexbin 19

hist2d 19

history 34

I

iloc 13

imports 4

index alignment 13

indexing 6

indicator 15

info 14

size encoding 19

slicing 6

small multiples 23

solutions 28

sort 10

split 7

split-apply-combine 17

split-apply-combine, agg, transform, filter, pivot tables 17

stack 7, 15

StringIO 13

structured array 10

stylesheet 21

subplots 21

surface plot 23

swaplevel 15

synthetic dataset 25

T

table of contents 2

take 10

ticks 21

transform 17

transformers 36

U

ufunc 8

ufuncs, broadcasting, aggregation, vectorization, performance 8

unstack 15

V

validate 15

vectorization 8, 26

views 6

visual hierarchy 23

visualization 19

W

weighted average 17

wireframe 23